

# Software Requirements Specification (SRS)

## Makerspace Management System

### 1. Introduction

#### 1.1 Purpose

The purpose of this document is to define the requirements for the Makerspace Management System (MMS) — a web-based application designed to streamline the operations of makerspace. The system will manage users, tools, projects, materials, and bookings efficiently while providing secure access and real-time data visibility.

#### 1.2 Scope

The Makerspace Management System will provide a centralized platform for managing user roles and permissions, registering and tracking equipment and tools, organizing projects, scheduling bookings, monitoring inventory, managing training sessions, and generating reports. It will consist of three main layers: Next.js frontend, Spring Boot backend, and PostgreSQL database.

#### 1.3 Definitions, Acronyms, and Abbreviations

| Term | Description                       |
|------|-----------------------------------|
| MMS  | Makerspace Management System      |
| DBMS | Database Management System        |
| API  | Application Programming Interface |
| JWT  | JSON Web Token                    |
| UI   | User Interface                    |
| CRUD | Create, Read, Update, Delete      |

#### 1.4 Technologies Used

- PostgreSQL 15+
- Spring Boot 3+
- Next.js 14+ with TypeScript and TailwindCSS
- Git & GitHub for version control

## 2. Overall Description

### 2.1 Product Perspective

The Makerspace Management System is a standalone web-based application that interfaces with a PostgreSQL database through RESTful APIs. It will be accessible via a standard web browser and hosted on a local or cloud-based server.

### 2.2 Product Functions

The system will support: user registration, equipment management, project tracking, booking scheduling, material inventory, training management, and reporting.

### 2.3 User Classes and Characteristics

| User Class         | Description                                                                                                                               |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Admin              | Has full system privileges — manages staff, users, tools, inventory, and reports.                                                         |
| Staff              | Includes both full-time workers and interns. Can manage bookings, assist in projects, and update inventory. Registered only by the admin. |
| Member/User        | Regular makerspace participant who can register online, book tools, and manage personal projects.                                         |
| Visitor (optional) | Can view general makerspace information but cannot interact with system data.                                                             |

### 2.4 Operating Environment

Frontend: Modern browsers (Chrome, Edge, Firefox)

Backend: Java 17+, Spring Boot

Database: PostgreSQL

Deployment: Local or hosted server

### 2.5 Constraints

- Internet or LAN connection required for access
- Staff registration restricted to admins
- Role-based access control enforced
- Limited file upload size if enabled

### 2.6 Assumptions and Dependencies

- Users will have valid credentials to log in
- PostgreSQL server must be active
- Spring Boot backend deployed before frontend access

### 3. System Features

#### User & Role Management

- Register new users with specific roles
- Admin-only staff registration (interns and full-time workers)
- Secure login with JWT authentication
- Profile management and access control

#### Tool & Equipment Management

- Add, edit, or remove tools from inventory
- Track availability and usage
- Log maintenance history
- Generate tool usage reports

#### Project Management

- Create and update projects
- Assign participants and supervisors
- Track milestones and completion status
- Attach project documents

#### Booking & Scheduling

- Book tools or workspace
- Prevent double-booking
- Admin approval workflow (if needed)
- View booking calendar and history

#### Inventory & Material Tracking

- Track materials and quantities
- Set restock alerts
- Link usage to projects

#### Training & Workshop Management

- Register and schedule training sessions
- Enroll participants

- Record attendance and completion

#### Reports & Analytics

- Generate usage statistics and summaries
- Project completion analytics
- Material and tool usage trends

### 4. Non-Functional Requirements

| Requirement Type | Description                                                  |
|------------------|--------------------------------------------------------------|
| Performance      | Average page load < 3 seconds; scalable for up to 500 users. |
| Security         | All endpoints protected with JWT and HTTPS.                  |
| Usability        | Clean, responsive interface using TailwindCSS.               |
| Reliability      | Daily backups and proper error handling.                     |
| Maintainability  | Modular, well-documented codebase.                           |
| Scalability      | Future integration with mobile and IoT systems.              |

### 5. Database Design

Entities:

- Users (user\_id, name, role, email, password)
- Tools (tool\_id, name, category, status, maintenance\_date)
- Projects (project\_id, title, owner\_id, status, start\_date, end\_date)
- Bookings (booking\_id, tool\_id, user\_id, date, duration, status)
- Inventory (item\_id, name, quantity, unit, threshold)
- Trainings (training\_id, title, instructor\_id, date, duration)

Relationships:

- One user can own multiple projects
- One tool can have multiple bookings
- One training can have many participants

### 6. System Architecture

The system uses a three-tier architecture:

1. Frontend (Next.js)
2. Backend (Spring Boot)

### 3. Database (PostgreSQL)

The frontend communicates with the backend via RESTful APIs secured with JWT tokens.

## 7. Future Enhancements

- QR code-based equipment checkout
- Real-time notifications
- Mobile app integration
- Automated maintenance reminders
- Docker and Jenkins integration for CI/CD deployment

## 8. References

- PostgreSQL Documentation
- Spring Boot Reference Guide
- Next.js Official Documentation
- TailwindCSS Documentation
- GitHub Resources